INSTITUTO FEDERAL FLUMINENSE - CAMPUS CAMPOS CENTRO

Carolina Souza

Lucas Travessa

ALGORITMO GENÉTICO

T2



Campos dos Goytacazes

2025

Sumário

Introdução **3**
A ideia **3**
1. Visão geral 3
2. Representação dos Indivíduos 4
3. Representação da População 4
Funcionamento **5**
1. Inicialização 5
2. Função de Avaliação 6
3. Cruzamento (Crossover) 7
4. Mutação 8
5. Função de elitismo 8
6. Gerações 9
7. Critério de Parada 10
Main **10**

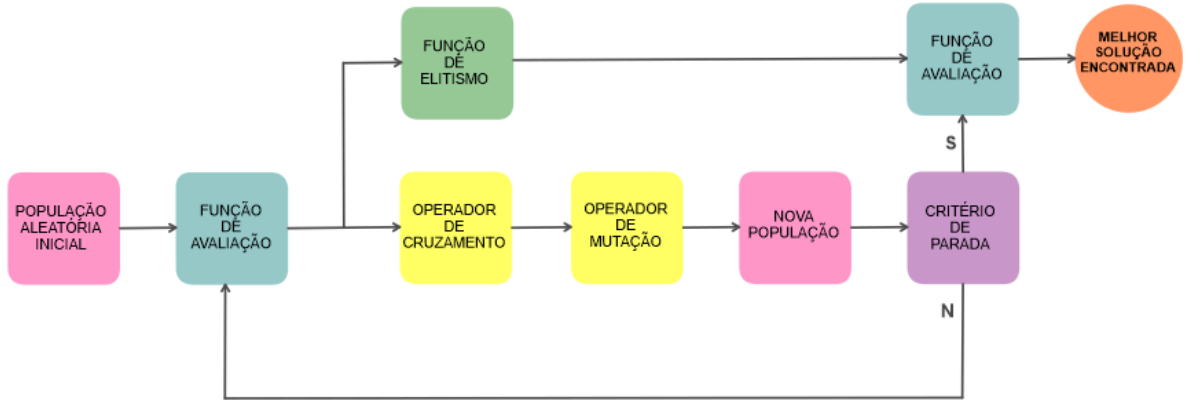
Introdução

O Algoritmo Genético (AG) é uma metaheurística de otimização e busca inspirados nos princípios da evolução natural, proposta por Charles Darwin. É um algoritmo populacional que pertence à classe dos Algoritmos Evolutivos. Desenvolvidos inicialmente por John Holland na década de 1960, os AGs simulam o processo de seleção natural, onde os indivíduos mais aptos têm maior probabilidade de sobreviver e reproduzir-se, passando suas características para as gerações futuras. Esses algoritmos são amplamente utilizados para resolver problemas complexos de otimização, especialmente aqueles que envolvem múltiplas variáveis e espaços de soluções de alta dimensionalidade.

A principal vantagem dos AGs é a sua capacidade de explorar grandes espaços de busca de forma eficiente, o que os torna robustos e versáteis. Eles são particularmente úteis em problemas onde outras técnicas de otimização falham, como em problemas NP-completos ou com múltiplos mínimos locais.

A ideia

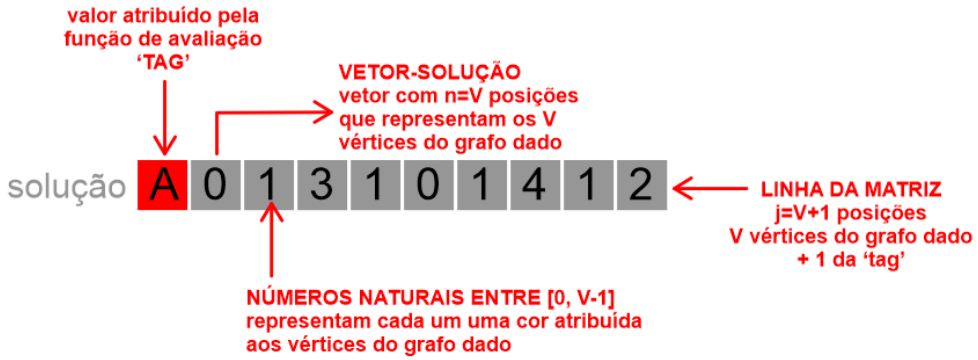
1. Visão geral



Fonte: Autoria própria

O funcionamento do algoritmo segue o padrão dos Algoritmos Genéticos estudados. Acontece uma geração aleatória de população inicial, e, a cada geração, aplicação da função de aptidão, passagem dos indivíduos pelos operadores genéticos para geração da população subsequente e novamente aplicação da função de aptidão. Nosso critério de parada é o número de gerações - 10 mil.

2. Representação dos Indivíduos



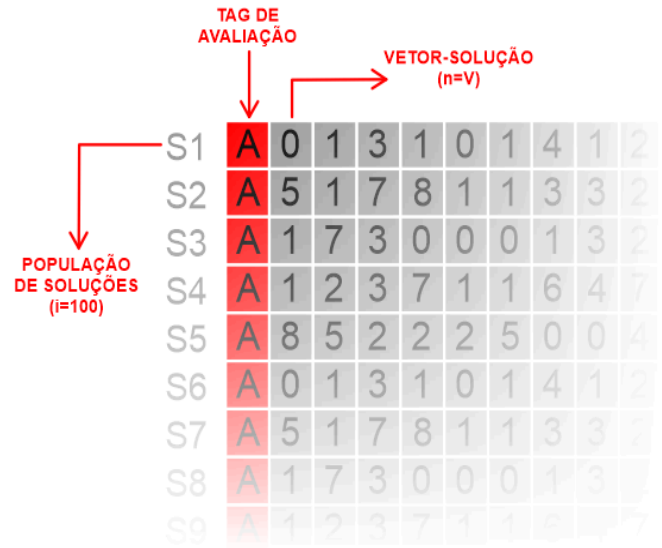
Fonte: Autoria própria

Os indivíduos são representados como vetores de tamanho V+1 de números naturais. A primeira posição não faz parte da solução em si, mas da tag gerada pela função de aptidão.

A posição S[0] contém o valor atribuído pela função de aptidão, que representa o número de cores utilizada naquela solução, quando ela for válida, e -1, quando a solução não representar uma coloração válida para o grafo analisado.

As posições de S[1] a S[V] fazem parte da solução propriamente dita, e cada posição representa um vértice do grafo e guarda um número natural que representa a cor daquele vértice.

3. Representação da População



Fonte: Autoria própria

A população é representada por uma matriz de 100 por V+1. Cada linha representa um indivíduo (solução), com o número fixo de 100. A primeira coluna armazena a tag da função de avaliação de cada indivíduo, enquanto as outras colunas (V colunas - número variável de acordo com o número de vértices do grafo avaliado) armazenam números naturais que representam cores dos vértices.

Funcionamento

1. Inicialização

```c
#include <math.h>

void gerar_solucao(int *solucao, int vertices)
{
    solucao[0] = 0; // Define a flag da solucao
    for (int i = 1; i < vertices + 1; i++)
    {
        solucao[i] = rand() % vertices; // Gera uma cor aleatória entre 0 e vertices-1
    }
}

void gerar_solucoes(int ***solucoes, int vertices, int num_solucoes)
{
    *solucoes = malloc(num_solucoes * sizeof(int *));
    for (int i = 0; i < num_solucoes; i++)
    {
        (*solucoes)[i] = malloc((vertices + 1) * sizeof(int));
        gerar_solucao((*solucoes)[i], vertices);
    }
}
```

A população inicial é gerada aleatoriamente, com cada indivíduo representando uma possível solução para o problema. O tamanho da população é um parâmetro importante, pois influencia a diversidade genética e a capacidade de exploração do espaço de busca. Nossa população foi fixada em 100 indivíduos.

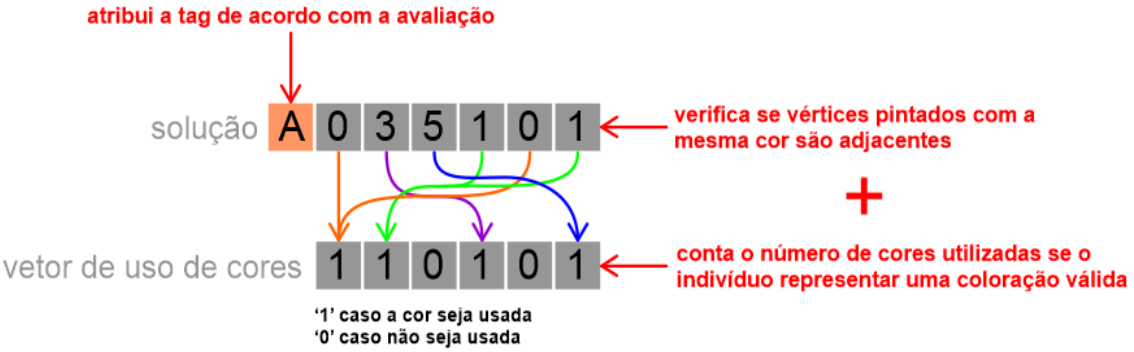
2. Função de Avaliação

A função de avaliação é o coração do AG, pois determina quão boa é uma solução. Ela verifica se vértices ‘pintados’ com a mesma cor são adjacentes e conta quantas cores existem em cada solução.

Caso encontre adjacência entre vértices de mesma cor (vértices aos quais foi atribuído o mesmo número natural), a função avalia a solução como inválida e coloca na posição de avaliação o valor -1.

Caso a solução seja válida, a função conta o número de cores presentes na solução e coloca na posição de avaliação o número de cores.

Quanto menor o número de cores de uma solução válida, melhor é essa solução.



Fonte: Autoria própria

```c
void funcao_avaliacao(int **mat, int vertices, int **solucao)
{
    for (int i = 0; i < vertices; i++)
    {
        for (int j = 0; j < vertices; j++)
        {
            if (i == j || (*solucao)[i + 1] != (*solucao)[j + 1])
                continue;
            if (mat[i][j] == 1)
            {
                // Vértices adjacentes com mesma cor => solução inválida
                (*solucao)[0] = -1; // Define a flag de erro na solução
                return;
            }
        }
    }
    int numero_cores = 0;
    int cores[vertices];
    for (int i = 0; i < vertices; i++)
    {
        cores[i] = 0; // Inicializa o vetor de cores
    }
    for (int i = 0; i < vertices; i++)
    {
        if (cores[(*solucao)[i + 1]] == 0)
        {
            cores[(*solucao)[i + 1]] = 1; // Marca a cor utilizada
            numero_cores++;               // Incrementa o número de cores utilizadas
        }
    }

    (*solucao)[0] = numero_cores; // Armazena o número de cores na primeira posição da solução
    return;
}
```

3. Cruzamento (Crossover)

O cruzamento combina os genes de dois indivíduos para gerar dois novos descendentes. Dois pontos de corte aleatórios são gerados e duas soluções são recombinadas, num cruzamento de dois pontos.



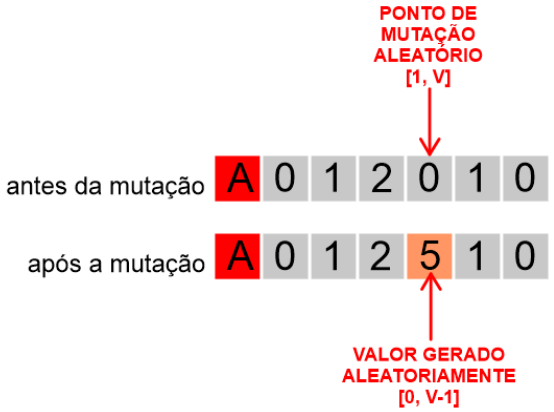
Cruzamento de dois pontos

Fonte: Autoria própria

```
3   void cruzamento(int **solucao1, int **solucao2, int vertices)
4   {
5       int pontoCorte1, pontoCorte2;
6       pontoCorte1 = 1 + (rand() % vertices);
7       pontoCorte2 = 1 + (rand() % vertices);
8
9       if (pontoCorte1 > pontoCorte2)
10      {
11          int aux = pontoCorte1;
12          pontoCorte1 = pontoCorte2;
13          pontoCorte2 = aux;
14      }
15
16      for (int i = 1; i < vertices + 1; i++)
17      {
18          if (i >= pontoCorte1 && i <= pontoCorte2)
19          {
20              int temp = (*solucao1)[i];
21              (*solucao1)[i] = (*solucao2)[i];
22              (*solucao2)[i] = temp;
23          }
24      }
25  }
```

4. Mutação

A mutação introduz diversidade na população, escolhendo uma posição aleatória no vetor de solução e introduzindo ali um valor aleatório tomado entre [0, V].



Fonte: Autoria própria

```c
void mutacao(int **solucao, int vertices)
{
    int pontoMutacao, cor;
    pontoMutacao = 1 + (rand() % vertices);

    cor = rand() % vertices;

    if ((*solucao)[pontoMutacao] != cor)
    {
        (*solucao)[pontoMutacao] = cor;
    }
}
```

5. Função de elitismo

Esta função armazena a melhor solução encontrada dentre todas as compostas em todas as gerações. A cada geração, as soluções são comparadas com a armazenada e a melhor, caso haja, é armazenada em substituição.

```c
void elitismo(int **populacao, int tamanhoPopulacao, int **melhorSolucao, int vertices)
{
    int indiceMelhor = 0, menorCores = vertices;
    for (int i = 0; i < tamanhoPopulacao; i++)
    {
        if (populacao[i][0] < menorCores && populacao[i][0] != -1)
        {
            menorCores = populacao[i][0];
            indiceMelhor = i;
        }
    }
    if (menorCores != -1 && menorCores < (*melhorSolucao)[0])
    {
        for (int j = 0; j <= vertices; j++)
        {
            (*melhorSolucao)[j] = populacao[indiceMelhor][j];
        }
    }
}
```

6. Gerações

Aqui, é gerenciada a aplicação dos operadores genéticos em cada geração.

```c
#include "./funcao-avaliacao.h"
#include "./mudanca-genetica.h"
#include <stdio.h>

int geracao(int **mat, int ***populacao, int vertices, int tamanhoPopulacao, int **melhorSolucao)
{
    int i;
    int menorNumCores = vertices;

    for (i = 0; i < tamanhoPopulacao; i++)
    {
        funcao_avaliacao(mat, vertices, &(*populacao)[i]);
    }

    elitismo(*populacao, tamanhoPopulacao, melhorSolucao, vertices);
    menorNumCores = (*melhorSolucao)[0];

    // Realiza o cruzamento e mutação para gerar nova população
    for (i = 0; i < tamanhoPopulacao; i++)
    {
        mutacao(&(*populacao)[i], vertices);
    }
    for (i = 0; i < tamanhoPopulacao; i++)
    {
        int temp = i;

        cruzamento(&(*populacao)[temp], &(*populacao)[++i], vertices);
    }

    return menorNumCores;
}
```

7. Critério de Parada

Nesta implementação, adotamos com o critério de parada o número fixo de 10000 gerações.

Main

```c
#include <stdio.h>
#include "./lidar-matriz.h"
#include "./gerar-solucoes.h"
#include "./geracao.h"

#define GERACOES 10000
#define TAMANHO_POPULACAO 10

int main()
{
    int **mat_adjacencia;
    int **solucoes;
    int vertices, arestas;

    if (carregar_matriz_ajacencia(&mat_adjacencia, &vertices, &arestas) != 1)
    {
        return 1;
    }

    int *melhor_solucao = malloc((vertices + 1) * sizeof(int));
    melhor_solucao[0] = vertices; // Inicializa com o número máximo de cores
    int menor_num_cores = vertices;

    printf("\nMatriz de adjacencia, %d vertices:\n", vertices);
    imprimir_matriz(mat_adjacencia, vertices);

    printf("\nGerando soluções aleatórias...\n");
    gerar_solucoes(&solucoes, vertices, TAMANHO_POPULACAO);


    printf("\nRodando por %d Geracoes\n", GERACOES);
    for (int i = 0; i < GERACOES; i++)
    {
        printf("\nGeracao %d\n", i + 1);
        menor_num_cores = geracao(mat_adjacencia, &solucoes, vertices, TAMANHO_POPULACAO, &melhor_solucao);
        printf("\nMenor numero de cores: %d\n", menor_num_cores);
    }

    printf("\nMenor numero de cores FINAL: %d", menor_num_cores);
    printf("\nMelhor solução encontrada FINAL:\n");
    for (int i = 1; i < vertices + 1; i++)
    {
        printf("%d, ", melhor_solucao[i]);
    }
    printf("\n");
    return 0;
}
```

Organiza as funções, assim como define o tamanho da população (100 indivíduos) e gerencia a sucessão das gerações (10000 gerações).

Funções relativas a matriz de Adjacência

```c
#include <stdio.h>
#include <stdlib.h>

int carregar_matriz_ajacencia(int ***mat, int *vertices, int *arestas)
{
    int i, j;
    FILE *arqEntrada = fopen("src/Grafo.txt", "r");

    if (arqEntrada == NULL)
    {
        printf("Erro ao abrir o arquivo.\n");
        return -1;
    }

    fscanf(arqEntrada, "%d %d", vertices, arestas);

    *mat = malloc(*vertices * sizeof(int *));
    for (i = 0; i < *vertices; i++)
        (*mat)[i] = malloc(*vertices * sizeof(int));

    for (i = 0; i < *vertices; i++)
    {
        for (j = 0; j < *vertices; j++)
        {
            fscanf(arqEntrada, "%d", &((*mat)[i][j]));
        }
    }

    fclose(arqEntrada);
    return 1;
}

void imprimir_matriz(int **mat, int vertices)
{
    for (int i = 0; i < vertices; i++)
    {
        for (int j = 0; j < vertices; j++)
        {
            printf("%d ", mat[i][j]);
        }
        printf("\n");
    }
}

void liberar_memoria_matriz(int **mat, int linhas)
{
    for (int i = 0; i < linhas; i++)
    {
        free(mat[i]);
    }
    free(mat);
}

void liberar_memoria_vetor(int *vetor)
{
    free(vetor);
}
```